A LiDAR Localization Method for Mobile Robots based on Fast Search and Outlier Mitigation

Ruilan Yang
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
yangrlan@mail2.sysu.edu.cn

Yuhua Qi*
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
qiyh8@mail.sysu.edu.cn

Hongbo Chen
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
chenhongbo@mail.sysu.edu.cn

Xiong Li
School of Systems Science and Engineering
Sun Yat-sen University
Guangzhou, China
lixiong6@mail.sysu.edu.cn

2025 11th International Conference on Control, Automation and Robotics (ICCAR) | 979-8-3315-2026-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAR64901.2025.11073039

***Abstract*—LiDAR localization gains more popularity by providing state information for robots in GPS-denied environments. Improving both the accuracy and computational efficiency is an ongoing effort for all LiDAR localization algorithms. In this paper, we propose a LiDAR localization method composed of a fast direct registration method and a sliding-window-based IMU propagator in a tightly coupled way. Also the factor graph optimization based on outlier mitigation is served as a back-end for robust state estimation. Our approach is extensively evaluated on multiple public datasets and also analyzed in the aspect of running time. Experimental results show that our system achieves high accuracy in various datasets while maintaining a real-time performance.**

***Index Terms*—LiDAR localization, estimation accuracy, real-time capability**

I. Introduction

Mobile robots are essential for achieving full automation in multiple regions, such as environment reconstruction [1], goods delivery [2] and disaster rescue [3]. To effectively perform these tasks, mobile robots require robust control and localization methods that enable precise navigation and interaction with their surroundings. Many efforts have been done to achieve accurate control utilizing different control techniques [4] and designing optimal controllers [5], [6]. Localization methods also have been broadly studied in the past two decades, which can be divided into visual and LiDAR branches. Visual localization is designed and provides six Degree-Of-Freedom (DOF) states using low-cost cameras but suffers in fast movements and unstable lighting conditions [7]. LiDAR localization is more stable for its high-precision perception of environments and resilience to brightness changes. With the decreasing costs of these LiDAR sensors, LiDAR localization is researched and deployed on various kinds of robots.

Numerous LiDAR localization systems have recently been proposed to achieve accurate pose estimation in complex environments. There are two main ways researchers work on to improve the accuracy and robustness in various scenarios. The first one is the processing of LiDAR scans. Direct registration is commonly adopted to process sufficient information from LiDAR point clouds. Numerous studies indicate that it gets better performance in texture-less and other extreme conditions than feature extraction methods [8]–[10]. The second one is the processing of IMU measurements and the global optimization. Factor graph optimization is utilized to propagate IMU measurements and maintain global consistency [11]. This optimization pattern is commonly more stable and accurate than filters as more historical states are introduced to benefit the current state estimation. However, the graph without outlier mitigation is more vulnerable when the state estimation is ill-conditioned. Besides the accuracy, the real-time capability of the localization system should not be neglected. Direct registration fully uses of the LiDAR scans to enhance the localization while increasing the time of pose estimation compared to the feature-selection methods. Filter-based methods tend to maintain high real-time capability but compromise accuracy, while optimization-based methods get the opposite. Thus it will be promising to integrate their advantages. To this end, we present a reliable and real-time LiDAR localization approach that provides accurate estimation and high efficiency for mobile robots. The main contributions are:

1) We propose a LiDAR-Inertial Odometry (LIO) providing precise localization information for mobile robots. The odometry utilizes the nanoflann tree to accelerate the search and adopts a sub-map aggregation strategy [8] to provide reliable correspondences.

2) We design a back-end optimization module to maintain the robustness of robot localization even with noisy sensor measurements. Specifically, the hessian of GICP [12] is decomposed to determine potential outliers, and Graduated Non-convexity (GNC) [13] is incorporated into the factor graph to correct the poses.

3) Comprehensive experiments on different datasets and scenarios are conducted to validate our system. Moreover, the real-time analysis is conducted to evaluate the computational efficiency of our method.

II. Related Work

Improving the accuracy and efficiency is a urgent need in the realm of localization. And this can be achieved by finding the accurate correspondences in two sets of the point clouds.

*Corresponding Author

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.

To find the correspondences, there are several approaches that can be categorized into two main branches:

1) Feature-based matching, which extracts the geometric and intensity features from the point cloud and estimates the transformation between the consecutive LiDAR scans. The pioneering LiDAR odometry such as LOAM [14], LEGO-LOAM [15], and LIO-SAM [16] are based on this method, which extracts the corner and edge features to estimate the robot's poses. T-LOAM extracts four kinds of geometric features and designs the corresponding residuals to obtain poses [17]. Intensity-SLAM extracts the extra intensity feature points and calculates the intensity residuals to enhance the pose estimation [18]. The feature extraction significantly reduces the computation time but may omit useful points due to specific extraction rules and suffer in the texture-less environments.

2) Direct registration, which uses the whole LiDAR scan as input and directly estimates the transformation using Iterative Closest Point (ICP) and its variants. Fast-LIO2 processes the whole scan and performs the registration using point-to-plane ICP [19]. DLIO registers raw point scans into local maps using Generalized Iterative Closest Point (GICP), improving the estimation accuracy [20]. The direct methods commonly have strong adaptability in diverse environments while consuming more time to deal with point correspondences.

Another way to improve the accuracy and efficiency is by designing the methods to fuse the IMU measurements with the LiDAR odometry during the estimation pipeline. Currently there are two fusion types: filter-based and optimization-based. Filter-based fusion utilizes the kalman filter and its variants to propagate the states. It predicts the prior state via IMU integration when receiving IMU measurements, then updates the posterior state using LiDAR odometry. LIO-EKF uses the Extended Kalman Filter (EKF) to fuse the measurements [21]. Fast-LIO2 and Point-LIO [22] are built upon this method using Iterated Error State Kalman Filter (IESKF), which incorporates the registration residuals into the model. The filter-based method demonstrates high computation efficiency, but it may encounter limitations such as estimation errors brought by linearization and high sensitivity to initial values. For graph-based methods, they employ factor graph optimization to fuse the sensor data and output the refined states. LIO-SAM formulates the odometry optimization based on the factor graph, incorporating different data sources such as IMU and point cloud into the pipeline [16]. Jonas et al. construct two types of GNSS factors and propose a robust graph that tightly fuses raw GNSS data with inertial measurements [23]. Bai et al. design an enhanced factor graph for simultaneous localization and calibration (SLAC) to integrate GNSS, IMU, and odometer measurements [24]. The factor graph is designed to handle the uncertainty and noise from the measurements, but processing a large volume of data and requires more computational resources. In this paper, the burden can be eased by designing the sliding-window-based IMU propagation without sacrificing the accuracy.

III. Methodology

In this section, a LiDAR localization method consisting of a LiDAR-inertial odometry and a factor graph is proposed, which achieves efficient localization and maintains global consistency simultaneously.

A. System Overview

The pipeline of our system is shown in Figure 1. The thread performs LIO to estimate the robot's poses with LiDAR scans and IMU measurements. The LIO starts by processing the received IMU data. An incremental fixed-lag smoother based on the theory of the sliding window is utilized to preintegrate the IMU measurements and output the prior states of the robot. The states also serve as an initial guess for the ongoing scan to map registration. Then the LiDAR scans are undistorted using the prior states and registered into a keyframe-based sub-map. Here, we employ an improved version of the GICP method for the registration called Nano-GICP, which accelerates the process by using the nanoflann tree structure. Also a hybrid aggregation strategy is introduced and modified here to fully utilize the keyframes. The estimated odometry information is sent to a factor graph for joint optimization and also updates the IMU propagator. The factor graph performs robust optimization with Graduated Non-convexity (GNC) and sends back the optimized odometry to the LIO module. Then, the incremental fixed-lag smoother repropagates using the optimization result and estimates the newest IMU biases for the next process.

B. LiDAR-Inertial Odometry

Our LIO includes two parts: the IMU pre-integration module to obtain the initial guess of poses, and the point cloud registration module that outputs the odometry.

1) *IMU Pre-Integration*

The IMU pre-integration module processes the raw IMU measurements including the angular velocity and linear acceleration. The measurement model of IMU can be depicted as:

$$\begin{aligned} \tilde{\boldsymbol{\omega}}_t &= \boldsymbol{\omega}_t + \mathbf{b}_t^{\omega} + \mathbf{n}_t^{\omega} \\ \tilde{\mathbf{a}}_t &= \mathbf{R}_t(\mathbf{a}_t + \mathbf{g}) + \mathbf{b}_t^{a} + \mathbf{n}_t^{a}, \end{aligned} \tag{1}$$

where $\tilde{\boldsymbol{\omega}}_t$ and $\tilde{\mathbf{a}}_t$ denote the raw angular velocity and linear acceleration, $\boldsymbol{\omega}_t$ and $\mathbf{a}_t$ are the actual values. The raw values contains biases including gyroscope bias $\mathbf{b}_t^{\omega}$ and accelerometer bias $\mathbf{b}_t^{a}$, along with the white Gaussian noises $\mathbf{n}_t^{\omega}$ and $\mathbf{n}_t^{a}$, respectively.

Using the measurement model, the typical IMU integration between two IMU timestamps is performed to get the position $\mathbf{p}_t$, velocity $\mathbf{v}_t$ and rotation matrix $\mathbf{R}_t$. To efficiently calculate the relative motion states between two LiDAR timestamps and update the result when the new IMU bias arrives, the pre-

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.

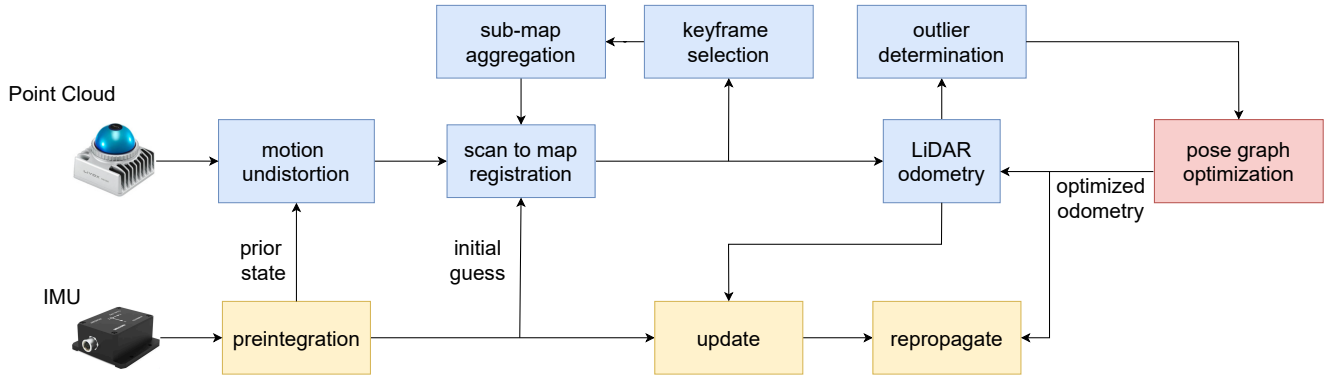


Fig. 1. The proposed system.

integration method is adopted. We define the preintegrated measurements between timestamp i and j as:

$$\begin{aligned}\Delta \boldsymbol{R}_{ij} &= \prod_{k=i}^{j-1} \operatorname{Exp}\left(\left(\tilde{\boldsymbol{\omega}}_k - \boldsymbol{b}_k^g - \boldsymbol{n}_k^g\right)\Delta t\right), \\ \Delta \boldsymbol{v}_{ij} &= \sum_{k=i}^{j-1} \Delta \boldsymbol{R}_{ik}(\tilde{\boldsymbol{a}}_k - \boldsymbol{b}_k^a - \boldsymbol{n}_k^a)\Delta t, \\ \Delta \boldsymbol{p}_{ij} &= \sum_{k=i}^{j-1} \left[\Delta \boldsymbol{v}_{ik}\Delta t + \frac{1}{2}\Delta \boldsymbol{R}_{ik}\left(\tilde{\boldsymbol{a}}_k - \boldsymbol{b}_k^a - \boldsymbol{n}_k^a\right)\Delta t^2\right],\end{aligned} \tag{2}$$

where Δt denotes the interval between the two timestamps.

IMU biases $\mathbf{b}_t^{\omega}$ and $\mathbf{b}_t^a$ are changed slowly and have to be estimated using factor graph optimization. The process is time-consuming with a large volume of IMU data. Inspired by Co-LRIO [25], we introduce the incremental fixed-lag smoother to propagate the IMU states, which is a type of graph using sliding window. This allows for a reduction in computational load and a shortening of computation time without affecting the localization accuracy.

2) *Point Cloud Registration*

This module aims to register the undistorted scan into a local map. Here we adopt and modify an aggregation strategy [8] to generate the local map. We perform a near search of the current pose based on the distance and obtain some neighbor keyframes. Then, we calculate the concave and convex hulls of the key poses and get several other keyframes corresponding to these poses. This step extracts keyframes far away from the current pose for more correspondences of the current scan. Different from other algorithms, considering the extreme cases such as rotation in place or too few frames, we also extract some of the newest keyframes. In this way, instead of the radius search performed in many algorithms, the geometric distance and the number of the correspondences can be simultaneously considered.

After determining the keyframe set, all selected frames are transformed into a global coordinate system using the previously estimated poses and aggregated to form a local map. We use the GICP algorithm to register the current scan $\boldsymbol{S}_k$ into the local map $\boldsymbol{M}_k$ and obtain the robot's pose $\boldsymbol{T}_{\boldsymbol{k}}$. To achieve high computational efficiency, we choose Nano-GICP for implementation. Specifically, for every point $\boldsymbol{p}$ in the scan, we search the correspondence $\boldsymbol{m}$ in the local map M_k using the nanoflann tree. The neighbors of points $\boldsymbol{p}$ and $\boldsymbol{m}$ are also collected for calculating the local covariances $\boldsymbol{C}_s$ and $\boldsymbol{C}_m$.

Using the correspondences and the transformation $\boldsymbol{T}_k$, the registration error $\boldsymbol{e}_i$ of each correspondence can be depicted as:

$$\boldsymbol{e}_i = \boldsymbol{m}_i - \boldsymbol{T}_k \boldsymbol{p}_i. \tag{3}$$

The cost function to calculate the final transformation is:

$$\boldsymbol{T} = argmin \sum_{i=1}^{N} \boldsymbol{e}_i^T (\boldsymbol{C}_m + \boldsymbol{T_k} \boldsymbol{C}_s \boldsymbol{T_k}^T)^{-1} \boldsymbol{e}_i, \tag{4}$$

where N is the number of points, $\boldsymbol{C}_s$ and $\boldsymbol{C}_m$ are the estimated covariances of the scan S_k and local map M_k, respectively. The Gauss-Newton method is applied to solve this problem, and the iteration is stopped until the registration error is less than a threshold or the iteration excels the maximum.

Inspired by [26], we decompose the hessian matrix $\boldsymbol{H}$ in the Gaussian-Newton iterations and get the smallest eigenvalue λ_{min} to determine the outlier. When the value is less than a threshold, the problem is ill-conditioned and we mark the estimated pose as a potential outlier. Then, the following factor graph optimization will be used to correct the poses.

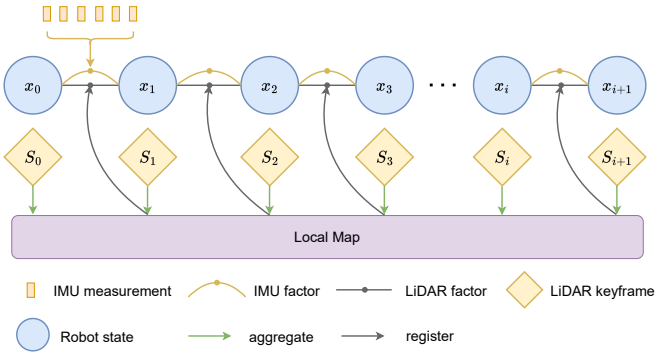


Fig. 2. The factor graph proposed in our system.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.

C. Factor Graph Optimization

The factor graph is shown in Figure 2, which fuses different kinds of factors, including a prior factor, IMU factors and LiDAR odometry factors.

The prior factor is set to the robot's initial pose at the beginning of the movement, and it changes to the prior pose after the factor graph marginalizes the old states.

Given the preintegrated measurements $\Delta \boldsymbol{R}_{ij}$, $\boldsymbol{v}_{ij}$ and $\Delta \boldsymbol{p}_{ij}$ between two timestamps i and j, the residual of the IMU factor can be described as:

$$\boldsymbol{r}_{ij}^{I} = \begin{bmatrix} \Delta \boldsymbol{R}_{ij}^{T}(\boldsymbol{R}_i^T \boldsymbol{R}_j) \\ \boldsymbol{R}_i^T(\boldsymbol{v}_j - \boldsymbol{v}_i - \boldsymbol{g}\Delta t) - \Delta \boldsymbol{v}_{ij} \\ \boldsymbol{R}_i^T(\boldsymbol{p}_j - \boldsymbol{p}_i - \boldsymbol{v}_i \Delta t - \frac{1}{2}\boldsymbol{g}\Delta t^2) - \Delta \boldsymbol{p}_{ij} \end{bmatrix}. \quad (5)$$

From the point cloud registration, we can obtain the relative pose $\Delta \boldsymbol{X}_{ij}$ between the robot states X_i and X_j, and form the odometry measurement with Gaussian noise $N(0, \sigma^2)$. Then the residual of the LiDAR factor is constructed as follows:

$$\boldsymbol{r}_{ij}^{L} = (\Delta \boldsymbol{X}_{ij} + N)^{-1}(\hat{\boldsymbol{X}}_i)^{-1}\hat{\boldsymbol{X}}_j, \quad (6)$$

where the superscript denotes the estimated poses.

Then the factor graph problem can be formulated with the above three types of factors:

$$\boldsymbol{X_f} = \arg\min_{X} \|\boldsymbol{r}^P\|^2 + \sum_{O} \|\boldsymbol{r}^O\|^2 + \sum_{I} \|\boldsymbol{r}^I\|^2, \quad (7)$$

where $\boldsymbol{X}_f$ is the final pose optimized by the factor graph.

D. Outlier Mitigation

To mitigate the effect of outliers, we utilize the smallest eigenvalue λ_{min} defined in Section III.B. According to the empirical eigenvalue thresholds, the Gaussian noise added to the LiDAR factors are divided into three types, which refer to well-localized, partial-localized and non-localized. Then the well-localized factors are classified and marked as inliers, while the other two types are seen as potential outliers. The Gaussian noise of the inliers is set to a small value denoting the high certainty.

Then we introduce the GNC mechanism into the factor graph to optimize the poses. The typical cost function of a non-least square (NLS) problem is defined as:

$$e = \sum_i w_i r_i^2(z_i, x), \quad (8)$$

where the function r_i denotes the residual of measurement z_i and the estimated states x, and w_i denotes the weight. To mitigate the impact of outliers, GNC modifies the NLS cost function with a robust kernel ρ and change the convexity of the cost function during the iteration. The cost function of GNC can be described as:

$$e = \sum_i w_i \rho(r_i^2) = \sum_i w_i r_i^2 + \Phi_\rho(w_i, \mu), \quad (9)$$

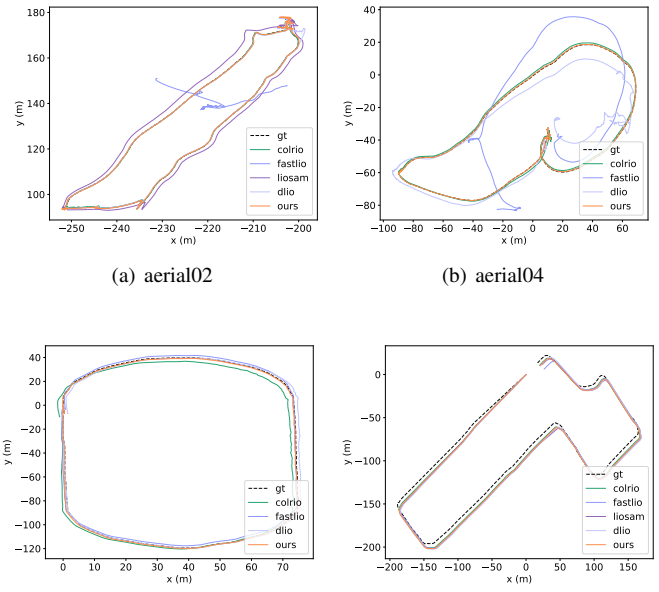
where Φ_ρ is the dual function, μ is the control parameter to change the shape of the function. And the robust kernel with the control parameter is:

$$\rho(x) = \frac{\mu c^2 x^2}{\mu c^2 + x^2}. \quad (10)$$

Readers can refer to [13] for more details. The ill-estimated poses are corrected during the optimization of GNC, which further reduces the error accumulation and improves the quality of the following optimization steps.

IV. Experiments

In this section, we evaluate the proposed LiDAR localization system in two public benchmark datasets, GRACO [27] and S3E [28]. GRACO is a dataset designed and captured by ground and aerial robots, while the unique air scenes and views pose challenges to the existing localization algorithms. S3E is a multi-modal dataset collected via a fleet of unmanned ground vehicles in a campus environment, featured in large-scale and scene diversity. In this paper, we set the same parameters for all experiments. In the point cloud registration module, the number of neighbor keyframes is set to 10, and the number of frames extracted from convex and concave hulls is set to 10. The eigenvalue thresholds to determine the level of localizability are set to 100 and 500, which indicates that a factor is seen as non-localized with the eigenvalue below 100. In the IMU propagation, the size of the sliding window is set to 10s. All the dataset experiments are conducted on an AMD R9 5900x CPU at 3.7GHz, an NVIDIA GeForce RTX 3060 GPU, and 32 GB memory.



(a) aerial02 (b) aerial04

(c) playground1 (d) campus1

Fig. 3. Trajectory estimates on various sequences compared with the ground truth.

A. Dataset Results

To validate the accuracy and efficiency of our system, we compare it against several current state-of-the-art and open-source systems. These include Fast-LIO2, LIO-SAM, DLIO and Co-LRIO. All algorithms are implemented on ROS1 except the Co-LRIO with a ROS2 implementation. Note that

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ACCURACY EVALUATION OF VARIOUS METHODS ON THE GRACO AERIAL DATASET.

Algorithm	Absolute Trajectory Error / Relative Trajectory Error (RMSE) (m)					
	01	02	03	04	05	06
Fast-LIO2	56.6736 / 1.4714	35.4212 / 1.0792	78.7773 / 1.3968	36.4546 / 0.9479	65.3522 / 2.0122	1.9332 / 0.2866
LIO-SAM	- / -	1.5438 / 0.1053	**0.1292** / 0.5517	- / -	- / -	0.6105 / 0.1603
DLIO	47.2515 / 0.4460	0.5831 / 0.0877	33.9218 / 0.2439	10.8520 / 0.1988	46.0482 / 0.3831	0.2256 / 0.0523
Co-LRIO	5.2904 / 0.1554	0.4465 / 0.0689	1.5293 / 0.0546	1.3393 / 0.0844	2.4844 / 0.0903	0.9650 / 0.1259
Ours	**1.2749** / **0.0873**	**0.2637** / **0.0468**	0.3628 / **0.0489**	**0.1967** / **0.0608**	**1.2154** / **0.0814**	**0.2394** / **0.0447**

'-' is used to mark the algorithm running failure.

we use the modified version Fast-LIO2 from DCL-SLAM [29] as the original implementation can not be run on the S3E dataset. We observed that the time consumption of the modified version is longer than the original one while the accuracy and robustness are improved. Thus we mainly focus on the accuracy comparison when referring to Fast-LIO2. Also the loop closure module of LIO-SAM is disabled as other methods don't perform the loop closure detection.

1) *GRACO Dataset*: We use the aerial sequences of GRACO dataset to evaluate the performance of our method. The results are shown in Table I presented in the Absolute Trajectory Error (ATE) and Relative Trajectory Error (RTE). The best results are displayed in bold while the second best results are underlined. The estimated trajectories of sequence-02 and sequence-04 are shown in Figure 3(a) and (b). When the aerial robot flies at a relatively low altitude, such as sequence-03, LIO-SAM outperforms other direct registration-based methods with richer features from the buildings and ground. Our method equipped with the sub-map aggregation strategy and robust optimization mitigates the drift and gets the second-best result. Looking at sequences collected at high altitudes, the point cloud is much sparser and texture-less. LIO-SAM completely fails in sequence-01 and sequence-04, due to its vulnerability to degenerated environment. Fast-LIO2 and DLIO cause extensive localization drift for the robot's rotation and vibration. Co-LRIO, which also incorporates the outlier mitigation, is observed sudden drift at the takeoff stage. Co-LRIO utilizes the intensity message for the second stage of registration, which may generate wrong correspondences in sparse cases. Our method successfully tracks the trajectories in all sequences regardless of the quick motion and degenerated environment. The ATE and RTE results also prove that our method outperforms other methods in terms of robustness and accuracy in such cases.

TABLE II
ACCURACY EVALUATION OF VARIOUS METHODS ON THE S3E DATASET.

Algorithm	Absolute Trajectory Error (RMSE) (m)			
	Playground1	Archway	Building	Campus1
Fast-LIO2	0.6924	0.8257	0.1635	4.0046
LIO-SAM	-	**0.4628**	0.4537	2.5962
DLIO	1.4750	0.5638	0.2008	3.2999
Co-LRIO	1.5500	0.6120	**0.1495**	**1.3077**
Ours	**0.4725**	0.5478	0.1520	1.3662

2) *S3E Dataset*: We evaluate the performance of the proposed method in the ground scenarios using S3E Dataset. The results are shown in Table II for all algorithms. In the building and archway sequences, the features provided by buildings and walls deliver strong constraints to LIO systems. Thus, the localization performance of our method is similar to others. LIO-SAM failed to track the trajectory in the playground sequence as there are few reliable features to extract. Fast-LIO2 and DLIO also have higher trajectory errors than our method, even though they take the whole point cloud for alignment. In contrast, the drift brought by sparse environmental features is suppressed by introducing outlier mitigation and our method obtains the best result. In Campus1 sequence, due to the vehicle passing over the feature-sparse bridge and traveling over a long distance, the localization errors of all methods exceed 1m and the inconsistency with the ground truth is obvious in Figure 3(d). In our method, the IMU propagation outputs the real-time IMU biases to provide precise motion priors and GNC-based optimization keeps the global consistency of the trajectory. This design helps our method to maintain a good performance in this sequence and obtain the second-best result.

B. Real-Time Analysis

Real-time capability is essential in localization systems as it have a great impact on the odometry frequency and the consumption of computational resources. Here we extract several sequences from S3E dataset for real-time analysis, as shown in the Table III. Thanks to the sliding window design of IMU propagation and the implementation of the fast-searching nanoflann tree, our method demonstrates an impressive real-time capability. In the Playground1 sequence with sparse environmental features, our method yields a high pose output rate of 16.21ms per frame. When the surrounding point clouds are relatively dense, such as in the Archway sequence, our method reduces the time by one-third compared to the second-fastest method Co-LRIO. The above analysis in different scenarios indicates that our method can reach an odometry frequency higher than 20Hz while maintaining robust localization performance.

V. CONCLUSIONS

In this paper we present a LiDAR localization method that tightly couples the fast direct registration with sliding-window-based IMU propagation. The framework also fuses with GNC-

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE TIME CONSUMPTION OF METHODS EVALUATED ON THE S3E DATASET.

Algorithm	Time consumption (ms)			
	Playground1	Archway	Building	Campus1
Fast-LIO2	28.6406	39.4634	29.3037	48.9219
LIO-SAM	-	32.4663	27.7181	35.7905
DLIO	99.8474	99.807	99.8335	99.8305
Co-LRIO	23.8687	30.6036	20.9522	34.429
Ours	**16.2103**	**20.9414**	**16.229**	**26.43**

based outlier mitigation for robust estimation. The method showcases outstanding localization performance in multiple scenarios while maintaining a good real-time capability. A series of experiments, including dataset tests collected on different robotic platforms, are conducted to show the robustness and accuracy of our method. In the future, the method will be extended to more applications and can be further utilized in multi-robot collaborative localization.

REFERENCES

[1] S. Lee, L. Chen, J. Wang, A. Liniger, S. Kumar, and F. Yu, "Uncertainty Guided Policy for Active Robotic 3D Reconstruction Using Neural Radiance Fields," IEEE Robotics and Automation Letters, vol. 7, no. 4, pp. 12070–12077, 2022, doi: 10.1109/LRA.2022.3212668.

[2] H. Li, H. Wang, C. Feng, F. Gao, B. Zhou, and S. Shen, "AutoTrans: A Complete Planning and Control Framework for Autonomous UAV Payload Transportation," IEEE Robotics and Automation Letters, vol. 8, no. 10, pp. 6859–6866, 2023, doi: 10.1109/LRA.2023.3313010.

[3] M. Arabboev, S. Begmatov, K. Nosirov, A. Shakhobiddinov, J. C. Chedjou, and K. Kyamakya, "Development of a Prototype of a Search and Rescue Robot Equipped with Multiple Cameras," in 2021 International Conference on Information Science and Communications Technologies (ICISCT), 2021, pp. 1–5. doi: 10.1109/ICISCT52966.2021.9670087.

[4] Chadi F. Riman and Pierre E. Abi-Char, "Fuzzy Logic Control for Mobile Robot Navigation in Automated Storage," International Journal of Mechanical Engineering and Robotics Research, Vol. 12, No. 5, pp. 313–323, September 2023.

[5] Fadwa Al-Momani, Asma Al-Tamimi, Ahmad Al-Jarrah, Ayat Al-Jarrah, and Mohammad Salah, "Tracking Control for a Two-Wheel Differentially Driven Nonholonomic Mobile Robot: Performance Comparison," International Journal of Mechanical Engineering and Robotics Research, Vol. 11, No. 10, pp. 724–736, October 2022. DOI: 10.18178/ijmerr.11.10.724-736.

[6] Salah M. Swadi, Ahmed K. Kadhim, and Ghusoon M. Ali, "Design of Path Planning Controller of Autonomous Wheeled Mobile Robot Based on Triple Pendulum Behaviour," International Journal of Mechanical Engineering and Robotics Research, Vol. 12, No. 1, pp. 23–31, January 2023. DOI: 10.18178/ijmerr.12.1.23-31.

[7] I. A. Kazerouni, L. Fitzgerald, G. Dooly, and D. Toal, "A survey of state-of-the-art on visual SLAM," Expert Systems with Applications, vol. 205, p. 117734, 2022, doi: https://doi.org/10.1016/j.eswa.2022.117734.

[8] K. Chen, B. T. Lopez, A. Agha-mohammadi, and A. Mehta, "Direct lidar odometry: Fast localization with dense point clouds," IEEE Robotics and Automation Letters, vol. 7, no. 2, pp. 2000–2007, 2022.

[9] Z. Chen, Y. Xu, S. Yuan, and L. Xie, "iG-LIO: An Incremental GICP-Based Tightly-Coupled LiDAR-Inertial Odometry," IEEE Robotics and Automation Letters, vol. 9, no. 2, pp. 1883–1890, 2024, doi: 10.1109/LRA.2024.3349915.

[10] Z. Yuan, F. Lang, T. Xu, and X. Yang, "SR-LIO: LiDAR-Inertial Odometry with Sweep Reconstruction," in 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2024, pp. 7862–7869. doi: 10.1109/IROS58592.2024.10802314.

[11] F. Lang, R. Ming, Z. Yuan, X. Xu, K. Wu, and X. Yang, "Adaptive Global Graph Optimization for LiDAR-Inertial SLAM," IEEE Robotics and Automation Letters, vol. 9, no. 11, pp. 9868–9875, 2024, doi: 10.1109/LRA.2024.3460413.

[12] A. Segal, D. Haehnel, and S. Thrun, "Generalized-icp.," in Robotics: science and systems, Seattle, WA, 2009, p. 435.

[13] H. Yang, P. Antonante, V. Tzoumas, and L. Carlone, "Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection," IEEE Robotics and Automation Letters, vol. 5, no. 2, pp. 1127–1134, 2020, doi: 10.1109/LRA.2020.2965893.

[14] J. Zhang, S. Singh, and others, "LOAM: Lidar odometry and mapping in real-time.," in Robotics: Science and systems, Berkeley, CA, 2014, pp. 1–9.

[15] T. Shan and B. Englot, "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2018, pp. 4758–4765.

[16] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in 2020 IEEE/RSJ international conference on intelligent robots and systems (IROS), IEEE, 2020, pp. 5135–5142.

[17] P. Zhou, X. Guo, X. Pei, and C. Chen, "T-LOAM: Truncated Least Squares LiDAR-Only Odometry and Mapping in Real Time," IEEE Transactions on Geoscience and Remote Sensing, vol. 60, pp. 1–13, 2022, doi: 10.1109/TGRS.2021.3083606.

[18] H. Wang, C. Wang, and L. Xie, "Intensity-slam: Intensity assisted localization and mapping for large scale environment," IEEE Robotics and Automation Letters, vol. 6, no. 2, pp. 1715–1721, 2021.

[19] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry," IEEE Transactions on Robotics, vol. 38, no. 4, pp. 2053–2073, 2022, doi: 10.1109/TRO.2022.3141876.

[20] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct lidar-inertial odometry: Lightweight lio with continuous-time motion correction," in 2023 IEEE international conference on robotics and automation (ICRA), IEEE, 2023, pp. 3983–3989.

[21] Y. Wu, T. Guadagnino, L. Wiesmann, L. Klingbeil, C. Stachniss, and H. Kuhlmann, "LIO-EKF: High frequency LiDAR-inertial odometry using extended kalman filters," in 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2024, pp. 13741–13747.

[22] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang, "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry," Advanced Intelligent Systems, vol. 5, Apr. 2023, doi: 10.1002/aisy.202200459.

[23] J. Beuchert, M. Camurri, and M. Fallon, "Factor Graph Fusion of Raw GNSS Sensing with IMU and Lidar for Precise Robot Localization without a Base Station," in 2023 IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 8415–8421. doi: 10.1109/ICRA48891.2023.10161522.

[24] S. Bai, J. Lai, P. Lyu, B. Wang, X. Sun, and W. Yu, "An Enhanced Adaptable Factor Graph for Simultaneous Localization and Calibration in GNSS/IMU/Odometer Integration," IEEE Transactions on Vehicular Technology, vol. 72, no. 9, pp. 11346–11357, 2023, doi: 10.1109/TVT.2023.3270424.

[25] S. Zhong et al., "CoLRIO: LiDAR-Ranging-Inertial Centralized State Estimation for Robotic Swarms," in 2024 IEEE International Conference on Robotics and Automation (ICRA), 2024, pp. 3920–3926. doi: 10.1109/ICRA57147.2024.10611672.

[26] J. Zhang, M. Kaess, and S. Singh, "On degeneracy of optimization-based state estimation problems," in 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 809–816. doi: 10.1109/ICRA.2016.7487211.

[27] Y. Zhu, Y. Kong, Y. Jie, S. Xu, and H. Cheng, "GRACO: A Multimodal Dataset for Ground and Aerial Cooperative Localization and Mapping," IEEE Robotics and Automation Letters, vol. 8, no. 2, pp. 966–973, 2023, doi: 10.1109/LRA.2023.3234802.

[28] D. Feng et al., "S3E: A Multi-Robot Multimodal Dataset for Collaborative SLAM," IEEE Robotics and Automation Letters, vol. 9, no. 12, pp. 11401–11408, 2024, doi: 10.1109/LRA.2024.3490402.

[29] S. Zhong, Y. Qi, Z. Chen, J. Wu, H. Chen, and M. Liu, "DCL-SLAM: A Distributed Collaborative LiDAR SLAM Framework for a Robotic Swarm," IEEE Sensors Journal, vol. 24, no. 4, pp. 4786–4797, 2024, doi: 10.1109/JSEN.2023.3345541.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 05:29:53 UTC from IEEE Xplore. Restrictions apply.